



Download ZIP from GitHub & Run Locally - Complete Guide

Here's how to download a GitHub project ZIP and get it running!



METHOD 1: Download ZIP from GitHub

Step 1: Download the ZIP

1. **Go to the GitHub repository**
 - Navigate to: <https://github.com/username/repository-name>
 2. **Click the Green "Code" Button**
 - Located near the top right of the file list
 3. **Click "Download ZIP"**
 - File downloads as: [repository-name-main.zip](#) or [repository-name-master.zip](#)
 4. **Save to your computer**
 - Choose a location (e.g., [~/Downloads/](#))
-

Step 2: Extract the ZIP

On Mac:

```
bash
```

```
# Navigate to Downloads
```

```
cd ~/Downloads
```

```
# Unzip the file
```

```
unzip repository-name-main.zip
```

```
# Rename folder (optional, removes -main suffix)
```

```
mv repository-name-main repository-name
```

```
# Navigate into folder
```

```
cd repository-name
```

On Windows:

bash

Right-click the ZIP file

Select "Extract All..."

Choose destination folder

Click "Extract"

Or using PowerShell:

Expand-Archive -Path "repository-name-main.zip" -DestinationPath "repository-name"

cd repository-name

On Linux:

bash

Navigate to Downloads

cd ~/Downloads

Unzip

unzip repository-name-main.zip

Navigate into folder

cd repository-name-main

Step 3: Install Dependencies

bash

Make sure you're in the project folder

pwd *# Should show path ending in /repository-name*

Install Node dependencies

npm install

This will:

- Read package.json

- Download all dependencies to node_modules/

- Can take 1-3 minutes depending on project size

If you get errors:

bash

Clear npm cache

```
npm cache clean --force
```

```
# Try again
```

```
npm install
```

```
# Or use yarn instead
```

```
yarn install
```

Step 4: Set Up Environment Variables

```
bash
```

```
# Check if .env.example exists
```

```
ls -la | grep env
```

```
# If .env.example exists:
```

```
cp .env.example .env
```

```
# If it doesn't exist, create .env manually:
```

```
touch .env
```

Edit .env file:

```
bash
```

```
# Open in your editor
```

```
nano .env
```

```
# or
```

```
code .env
```

```
# or
```

```
vim .env
```

Add your credentials:

```
bash
```

```
# Supabase
```

```
VITE_SUPABASE_URL=https://your-project.supabase.co
```

```
VITE_SUPABASE_ANON_KEY=your_anon_key_here
```

```
# Gemini AI
```

```
VITE_GEMINI_API_KEY=your_gemini_api_key_here
```

Save and close (Ctrl+X, then Y, then Enter in nano)

Step 5: Run the Development Server

bash

Start the dev server

`npm run dev`

You should see output like:

VITE v5.0.0 ready in 500 ms

→ Local: http://localhost:5173/

→ Network: use --host to expose

Step 6: Open in Browser

bash

Automatically open browser (Mac):

`open http://localhost:5173`

Automatically open browser (Windows):

`start http://localhost:5173`

Automatically open browser (Linux):

`xdg-open http://localhost:5173`

Or just manually navigate to:

`http://localhost:5173`

METHOD 2: Git Clone (Better for Development)

Step 1: Clone the Repository

bash

Navigate to where you want the project

`cd ~/Projects` *# or wherever you keep code*

Clone with git

`git clone https://github.com/username/repository-name.git`

Navigate into folder

`cd repository-name`

Step 2-6: Same as Method 1

`bash`

Install dependencies

`npm install`

Set up .env

`cp .env.example .env`

Edit .env with your keys

Run dev server

`npm run dev`

Benefits of Git Clone:

-  Can pull updates easily: `git pull`
 -  Can contribute back
 -  Full git history
 -  Can create branches
-



COMMON ISSUES & FIXES

Issue 1: "npm: command not found"

Cause: Node.js not installed

Fix:

`bash`

Install Node.js from nodejs.org

Or using Homebrew (Mac):

`brew install node`

Or using apt (Linux):

`sudo apt install nodejs npm`

Verify installation:

`node --version`

`npm --version`

Issue 2: "Port 5173 already in use"

Cause: Another process using port 5173

Fix:

bash

Find and kill process on port 5173

Mac/Linux:

`lsof -ti:5173 | xargs kill -9`

Windows:

`netstat -ano | findstr :5173`

Note the PID, then:

`taskkill /PID <PID> /F`

Or use a different port:

`npm run dev -- --port 3000`

Issue 3: "Module not found" errors

Cause: Dependencies not installed properly

Fix:

bash

Delete node_modules and package-lock.json

`rm -rf node_modules package-lock.json`

Reinstall

`npm install`

If still failing, check Node version

`node --version`

Should be 18.x or higher

Update Node if needed:

Using nvm (recommended):

`nvm install 18`

`nvm use 18`

Issue 4: ".env file not loading"

Cause: Wrong variable names or file not in root

Fix:

bash

Verify .env is in project root

`ls -la .env`

Check variable names start with VITE_

Vite only loads variables starting with VITE_

Restart dev server after editing .env

Stop server: Ctrl+C

`npm run dev`

Issue 5: "Supabase connection failed"

Cause: Wrong Supabase credentials

Fix:

1. Go to Supabase dashboard
 2. Project Settings → API
 3. Copy exact URL and anon key
 4. Update .env file
 5. Restart server
-

Issue 6: Blank page / White screen

Cause: Build errors or missing environment variables

Fix:

bash

Check browser console for errors

Right-click → Inspect → Console tab

Common fixes:

1. Check .env file exists and has values

cat .env

2. Rebuild

npm run build

npm run preview

3. Check for TypeScript errors

npm run type-check

4. Clear browser cache

Hard reload: Cmd+Shift+R (Mac) or Ctrl+Shift+R (Windows)



COMPLETE SETUP CHECKLIST

bash

✓ Prerequisites:

- ☐ Node.js 18+ installed
- ☐ npm or yarn installed
- ☐ Code editor (VS Code, etc.)

✓ Download & Extract:

- ☐ Downloaded ZIP from GitHub
- ☐ Extracted to local folder
- ☐ Navigated into project directory

✓ Install Dependencies:

- ☐ Ran: npm install
- ☐ No errors during install
- ☐ node_modules/ folder created

✓ Environment Setup:

- ☐ Created .env file
- ☐ Added Supabase URL
- ☐ Added Supabase anon key
- ☐ Added Gemini API key

✓ Run Development Server:

- ☐ Ran: npm run dev
- ☐ No errors in terminal

☐ Server running on port 5173

☒ Verify in Browser:

☐ Opened http://localhost:5173

☐ Page loads (not blank)

☐ No errors in browser console

☐ Features work as expected

☒ Test Features:

☐ Can navigate between pages

☐ Can use simulator

☐ Can send messages to AI chat

☐ Integrations page loads

QUICK REFERENCE COMMANDS

bash

Download and setup (full process)

unzip repository-name.zip

cd repository-name

npm install

cp .env.example .env

Edit .env with your keys

npm run dev

Daily development workflow

cd repository-name *# Navigate to project*

git pull *# Get latest changes (if using git)*

npm run dev *# Start dev server*

Building for production

npm run build *# Creates dist/ folder*

npm run preview *# Preview production build*

Troubleshooting

rm -rf node_modules package-lock.json

npm install *# Fresh install*

npm run dev -- --port 3000 *# Use different port*

SECURITY REMINDER

Before sharing your downloaded project:

```
bash
```

```
# Make sure .env is in .gitignore
```

```
cat .gitignore | grep .env
```

```
# Should see:
```

```
# .env
```

```
# .env.local
```

```
# .env.development.local
```

```
# Check what would be committed
```

```
git status
```

```
# Should NOT see .env in the list
```

```
# If .env appears:
```

```
echo ".env" >> .gitignore
```

```
git rm --cached .env
```

```
``
```

```
--- end ---
```

```
## 💡 PRO TIPS
```

```
#### **Tip 1: Use a Git Client (Optional)**
```

```
- **GitHub Desktop** - Easy GUI
```

```
- **Sourcetree** - More features
```

```
- **VS Code** - Built-in Git
```

```
#### **Tip 2: Use VS Code Extensions**
```

```
``
```

```
- ESLint - Code quality
```

```
- Prettier - Code formatting
```

```
- Tailwind CSS IntelliSense - CSS autocomplete
```

```
- TypeScript - Better type support
```

Tip 3: Hot Module Replacement

```
bash
```

```
# Vite automatically reloads on file changes
```

Just save your files and see changes instantly!

Tip 4: Environment Variables

bash

Access in code:

```
import.meta.env.VITE_SUPABASE_URL
```

NOT process.env - that's for Node.js

Vite uses import.meta.env



YOU'RE ALL SET!

Now you can:

-  Download any GitHub project as ZIP
 -  Extract and set it up locally
 -  Install dependencies
 -  Configure environment
 -  Run development server
 -  Make changes and see them live
 -  Troubleshoot common issues
-



NEXT STEPS

1. **Make changes to the code**
2. **Test locally**
3. **Build for production:** `npm run build`
4. **Deploy** (Vercel, Netlify, etc.)